Introduction to C++

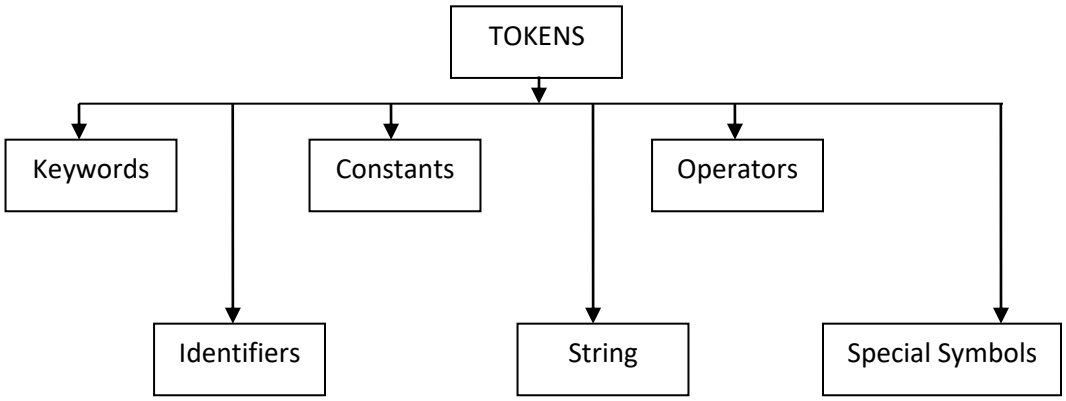
The C++ programming language was developed at **AT & T** Bell Laboratories in the early 1979-80s by ***Bjarne Stroustrup*.** C++ is a general-purpose **object-oriented programming (OOP)** language, and is an extension of the C language.

C++ Character Sets:

Letters	A-Z , a-z
Digits	0-9
Special Symbols	Space + - * / ^ \ ( ) [ ] { } = != <> . „ “ $, ; : % ! & ? _ # <= >= @
White Spaces	Blank spaces, horizontal tab, carriage return
Other Characters	Any of the 256 ASCII character

TOKENS

The smallest individual units in a program are known as token. In simple terms, a C++ program is a collection of tokens, comments, and white space. Tokens used in C++ are:



KEYWORDS. Keywords are certain reserved words that convey a special meaning to the compiler. These are reserve for special purpose and must not be used as identifier name.eg **for , if, else , this , do, while, public, private protected, virtual etc.**

IDENTIFIER: An **identifier** is a name that is assigned by the user for a program element such as variable, type, template, class, function. C++ identifier follows the following rules.

- **i.** They must not start with digits. (**2a, 1stName Invalid**)
- **ii.** Uppercase Lowercase letters are distinct (**A, a**)

iii. Never Use C++ keywords as Identifiers. **(break, while etc are not allowed)**
iv. They can have Alphabet, digit and Underscore (**_MIA2**)
v. Alpha Numeric Character are allowed as Identifier (**JH94A**)

.

List of Following Valid Identifiers
MyFile
Date_7_9
_Chk
_DS
_HK_L

List of Following Inalid Identifiers
29abc
break
My.File
Data-Rec
while

Question: Find the correct Identifier out of the following, which can be used for naming variables, constants, or function name in C++ program.

For, while , INT , New , delete , 1stName , Add+Subtract , Name1 , While , for , Float , A%B, _C

CONSTANTS

The data items which never change their value throughout the program Execution. There are several kinds of Constants:

i. **Integer Constants**
ii. **Character Constants**
iii. **Floating Constants**
iv. **String Constants**

i. **Integer Constants**: Integer Constants are whole numbers without any fractional part. An integer literal must have at least one digit and must not contain any decimal point.

   a. **Decimal Integer Constants:-** An integer literal without leading 0 (zero) is called decimal integer Constants e.g., 123, 786 , +97 , etc.

   b. **Octal Integer Constants:-** A sequence of octal digit starting with 0 (zero) is taken to be an octal integer literal (zero followed by octal digits). **e.g., 0345, 0123, etc.**

   c. **Hexadecimal Integer Constants** :- Hexadecimal Integer Constants starts with 0x or 0X followed by any hexa digits. **e.g., 0x9A45, 0X1234, etc.**

ii. **Character Constants :** Any single character enclosed within single quotes is a character literal. **e.g ‘ A’ , ‘3’.**

iii. **Floating Constants :** Numbers which are having the fractional part are referred as floating Constants or real Constants. It may be a positive or negative number. A number with no sign is assumed to be a positive number. **e.g 2.0, 17.5, -0.00256.**

iv. String Constants : It is a sequence of character surrounded by double quotes. **e.g., "abc" , "23".**

OPERATORS

Operators are used in program to manipulate data and variables. They are usually form part of the mathematical and logical expression. C++ operator can be classified into a eight categories:

- **i.** Arithmetic Operator
- **ii.** Relational Operator
- **iii.** Logical Operator
- **iv.** Assignment Operator
- **v.** Increment | Decrement Operator
- **vi.** Conditional Operator
- **vii.** Bitwise Operator
- **viii.** Special Operator

i. Arithmetic Operator: C++ provides all the basic arithmetic operator. There can be two types of operator

- **a. Unary arithmetic operator** : Unary arithmetic operator Require only one operand. Example : -a, +a , a++ , --a
- **b. Binary arithmetic operator** : Binary arithmetic operator Require two operands. Example : a+b, a-b. There are five Binary arithmetic operator **+, - , * , / , %**

ii. Relational Operator: Relational Operator is used to compare values of two expression, and return **'1'** if it is true or return **'0'** if it is false. **(<, <=, > , >= ,!=, ==)**

iii. Logical Operator: The logical operators are AND (&&), OR (||) and NOT (!). They are used to combine two different expressions together. **(&& ,|| and !)**

iv. Assignment Operator : Operates '=' is used for assignment, it takes the right-hand side and copy it into the left-hand side. **(+= , -= ,*= , /= , %=)**

v. Increment | Decrement Operator : These operators are used to either increase or decrease the value of the variable by one. There are two types of increment decrement operators.

Pre Increment	**: ++a (a=a+1)**	**Pre Decrement**	**: - --a (a=a-1)**
Post Increment	**: a++ (a=a+1)**	**Post Decrement**	**: a-- (a=a+1)**

e.g. 5++ (Illegal Declaration)

vi. Conditional Operator : Conditional operators return one value if condition is true and returns another value is condition is false. This operator is also called as ternary operator.

Syntax : (Condition ? true_value : false_value);

WAP find largest of two numbers.

```
main()
{
int a,b,max=0;
clrscr();
cout<<"Enter First Number "<<endl;
cin>>a;
cout<<"Enter Second Number "<<endl;
cin>>b;
max =( a > b ? a : b ) ;
cout<<"Largest = "<<max;
getch();
}
```

WAP find largest of three numbers.

```
main()
{
int a,b,c, max=0;
clrscr();
cout<<"Enter First Number "<<endl;
cin>>a;
cout<<"Enter Second Number "<<endl;
cin>>b;
cout<<"Enter Third Number "<<endl;
cin>>c;
max = a > b ? (a > c ? a : c) : (b > c ? b : c) ;
cout<<"Largest = "<<max;
getch();
}
```

vii. Bitwise Operator : These operators are used to perform bit operations. Decimal values are converted into binary values which are the sequence of bits and bit wise operators work on these bits.

& (Bitwise AND) , | (Bitwise OR) , >> (Bitwise Right Shift), << (Bitwise Left Shift), ~ (Bitwise Inversion (NOT))

viii. Special Operator : Apart from the above operators there are some other operators available in C++ used to perform some specific task. Some of them are discussed here: sizeof operator , comma (,) operator.

```
main()
{
  int a=5 , b;
  b=sizeof(a)
  cout<<b;
} output : 2 Bytes
```

```
Sizeof ( int)      =   2  byte
Sizeof (float)     =   4  byte
Sizeof (char)      =   1  byte
Sizeof (double)    =   8  byte
```

STRING

Sequence of Characters are called string. Strings are regarded as an array of characters, ending with a null character **"\0"**. The null character is used to point the ending of the specified string. Strings, in practical use are expected to be enclosed in double quotes (" ").

"JAIPUR", "TONK", "NIWAI"

SPECIAL SYMBOLS

The symbols that are used in C++ with some special meaning and for some specific functions are called as Special Symbols. The special symbols being used with context to programming language are illustrated as: **[] , {} , ; , () , *, & , # , =**

ESCAPE SEQUENCES

Is a non-printing character. All escape sequences consist of two or more characters, the first of which is the backslash, \ (called the "Escape character"); the remaining characters determine the interpretation of the escape sequence. Example: **“\n”**

`\'`	**single quote**
`\"`	**double quote**
`\?`	**question mark**
`\\`	**backslash**
`\a`	**audible bell**
`\b`	**backspace**
`\f`	**form feed - new page**
`\n`	**line feed - new line**
`\r`	**carriage return**
`\t`	**horizontal tab**
`\v`	**vertical tab**

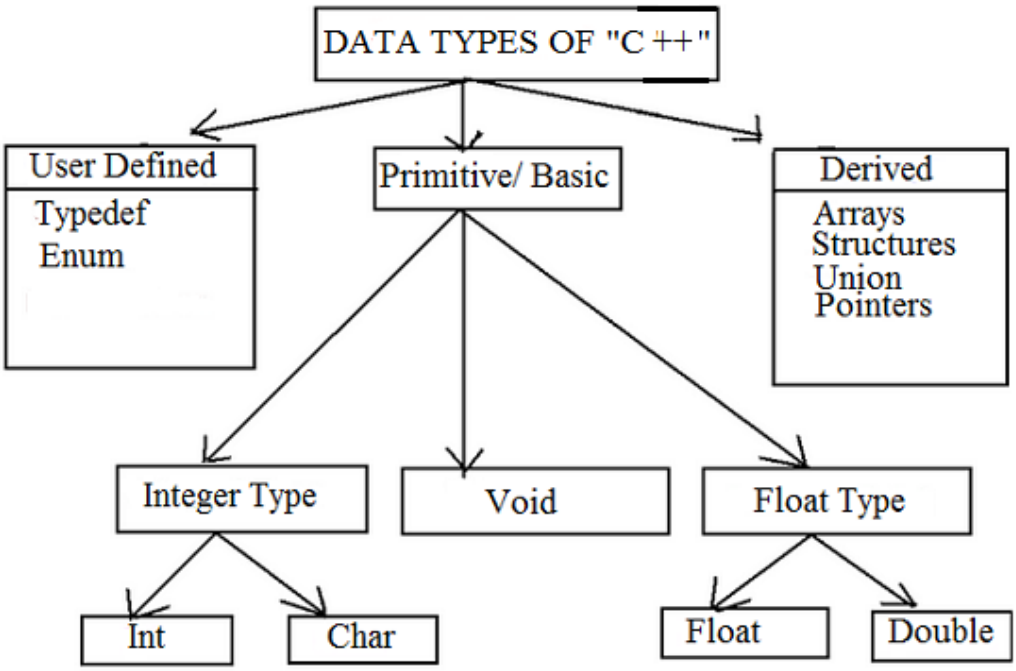
Data Type Modifiers

As the name implies, data type modifiers are used with the built-in data types to modify the length of data that a particular data type can hold. Data type modifiers available in C++ are:

Signed
Unsigned
Short
Long

DATA TYPES IN C++

All variables use data-type during declaration to restrict the type of data to be stored. Therefore, we can say that data types are used to tell the variables the type of data it can store. Data types in C++ is mainly divided into Three categories.:**Primitive Data Type , Derived Data Type and User Defined Data Type.**



Primitive or Fundamental or Built-in data types: These data types are already known to compiler. These are the data types those are not composed of other data types. There are following fundamental data types in C++: int , float , double , char and void.

Derived Data Type : These are the data types that are composed of fundamental data types. e.g., array, class, structure, etc.

Variable Type	Keyword	Bytes Required	Range
Character (signed)	Char	1	-128 to +127
Integer (signed)	Int	2	-32768 to +32767
Float (signed)	Float	4	-3.4e38 to +3.4e38
Double	Double	8	-1.7e308 to + 1.7e308
Long integer (signed)	Long	4	2,147,483,648 to 2.147.438.647
Character (unsigned)	Unsigned char	1	0 to 255
Integer (unsigned)	Unsigned int	2	0 to 65535
Unsigned long integer	unsigned long	4	0 to 4,294,967,295
Long double	Long double	10	-1.7e932 to +1.7e932

COMMENTS IN A C++ PROGRAM.

Comments are the section of code that compiler ignores or skip to compile. There are two types of comments in C++.

1. Single line comment: The comments that begin with // are single line comments. The Compiler simply ignores everything following // in the same line.

2. Multiline Comment : The multiline comment begin with **/*** and end with ***/** . This means everything that falls between **/*** and ***/** is consider a comment even though it is spread across many lines. **e.g.**

```
 main ()
{
clrsccr();
cout << " hello world";   // Single Line Comment
/* this is the program to print hello world
For demonstration of comments.  */
getch();   }
```

TYPE CONVERSION

Type Conversion:-The process of converting one predefined data type into another is called type conversion. C++ facilitates the type conversion in two forms:

(i) Implicit type conversion (Automatic):- An implicit type conversion is a conversion performed by the compiler without programmer's intervention. An implicit conversion is applied generally whenever different data types are intermixed in an expression. The C++ compiler converts all operands upto the data type of the largest data type's operand, which is called type promotion.

```
main()
{
int a=5,b=2, c=0;
clrscr();
c= a / b;       //   Implicit type conversion
cout<< c;
getch();
}
```

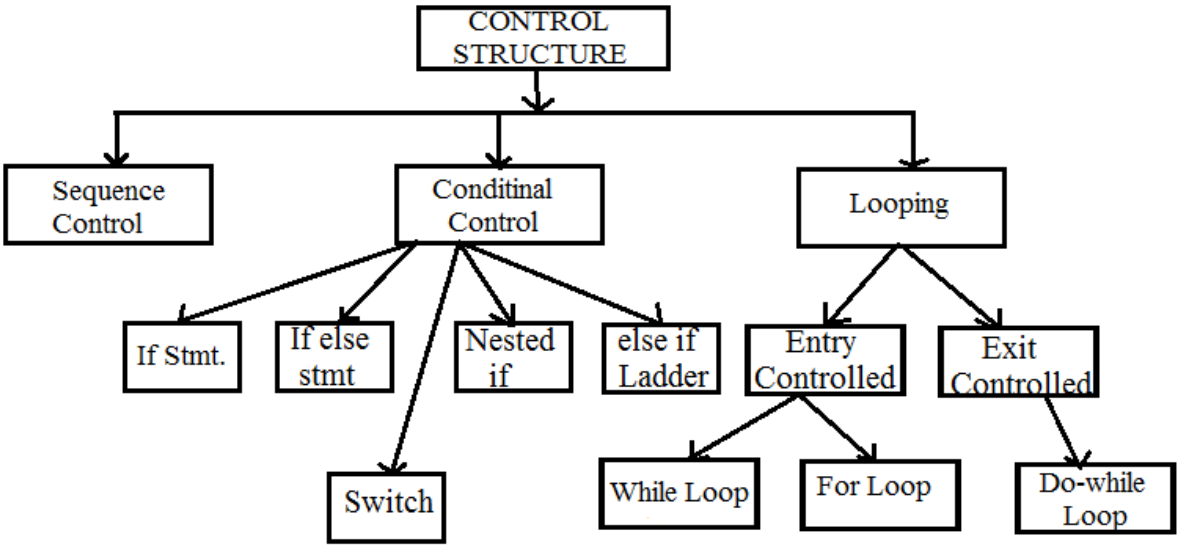
Output : 2 [Output automatically converted to integer]

(ii) Explicit type conversion: - An explicit type conversion is user-defined that forces an expression to be of specific data type. The explicit conversion of an operand to a specific type is called type casting.

```
main()
{
int a=300;
char b;
clrscr();
b= a      //   Explicit type conversion
cout<< b;
getch();
}
```
Output : 44

CONTROL STATEMENTS

C++ Control statements control the order of execution in a C++ program, based on data values and conditional logic. There are four main categories of control flow statements.



1. **Sequence Control :** The sequence construct means the statements are being executed sequentially. It represents the default flow of statements. Ex: WAP display your name.
2. **Selection (Branching) statements:** The selection construct means the execution of statement(s) depending on a condition. If a condition is true, a group of statements will be execute otherwise another group of statements will be execute. Example

if (<conditional expression>)

<statement action> TRUE

The if statement executes a block of code only if the specified expression is true. If the value is false, then the if block is skipped and execution continues with the rest of the program.

if (<conditional expression>)
<statement action> TRUE
else
<statement action> FALSE

The if…. else statement is an extension of the if statement. If the statements in the if statement fails, the statements in the else block are executed.

switch (<Condition >)
{
case Label :
break;
case Label :
break;
default:
} The switch case statement, also called a case statement is a multi-way branch with several choices.

3. **Loop statements:** Looping the iteration construct means repetition of set of statements depending upon a condition test. The iteration statements allow a set of instructions to be performed repeatedly until a certain condition is true.

while(<conditional expression >)
{
<statement action>
Increment | Decrement
}
The loop repeatedly execute while the loop_condition evaluates to true. When the loop_condition becomes false, the program control passes to the statement after the loop_body.

do
{
<statement action>
Increment | Decrement
} while(<condition>) ;

do-while loop is an exit-controlled loop i.e. it evaluates its loop_condition at the bottom of the loop after executing its loop_body statements. It means that a do-while loop always executes at least **once.**

for (initialization_expression ; loop_Condition ; Increment | Decrement)
{
<statement action>
}
Working of the for Loop:-
1. The initialization_expression is executed once, before anything else in the for loop.
2. The loop condition is executed before the body of the loop.
3. If loop condition is true then body of loop will be executed.
4. The **Increment | Decrement** is executed after the body of the loop
5. After the **Increment | Decrement** is executed, we go back and test the loop condition again, if loop_condition is true then body of loop will be executed again, and it will be continue until loop_condition becomes false.

Difference Between **Entry Control Loop** and **Exit Control Loop**

Jumping : These statements unconditionally transfer control within function . In C++ four statements perform an unconditional branch :

1. return
2. goto
3. break
4. continue

1. **return Statement:-** The return statement is used to return from a function.

2. **goto statement** :- A goto Statement can transfer the program control anywhere in the program. The target destination of a goto statement is marked by a ***label***.

```
main()
{
  int a=3,b=4;
  clrscr();
  if ( a <  b )
  goto dumka;
  cout << " Hello " << endl;
  cout << " Thank You " << endl;
  dumka:
  cout << " I am Dumka " ;
  getch();
}
```

3. **break Statement** :- The break statement enables a program to exit from current loop. (refer to *switch case* program)
4. **continue keyword** : The *continue statement* is somewhat different from break. Instead of forcing termination, it forces the next iteration of the loop to take place, skipping any code in between.

   WAP find Prime Factor of any number using continue.

```
main()
{
 int n,i=2;
 clrscr();
 cout<< “Enter The Number”<<endl;
 cin>>n;
 while(n>1)
 {
  if(n%i==0)
  cout<<i<<endl;
  else
  {
   i++;
   continue;
  }
  n=n/2;
 }
 getch();
}
```

Array

An array is a collection of data items, all of the same type, accessed using a common name. where indexing start from 0 to n-1. The number of subscript or index determines dimensions of the array. There are mainly two types of array.

a. One Dimensional Array (Vector)
b. Two Dimensional Array. (Matrix)

One Dimensional Array : The one dimensional array or single dimensional array in C++ is the simplest type of array that contains only one row for storing data. It has single set of square bracket ("[ ]"). It can be represented as **int roll[8] = { 12 , 45 , 32 , 23 , 17 , 49 , 5 , 11 }**

Name	roll[0]	roll[1]	roll[2]	roll[3]	roll[4]	roll[5]	roll[6]	roll[7]
Values	12	45	32	23	17	49	5	11
Address	1000	1002	1004	1006	1008	1010	1012	1014

1-D Array memory arrangement

Two Dimensional Array: The two-dimensional array in C++ is such type of array that contains more than one row and column i.e. Table, to store data on it. The two-dimensional array is also known as a Matrix array in C++. Representation of **A [4] [4]** array **(A[row] [col])**

	Col 1	Col 2	Col 3	Col 4
Row 1	0, 0	0, 1	0, 2	0, 3
Row 2	1, 0	1, 1	1, 2	1, 3
Row 3	2, 0	2, 1	2, 2	2, 3
Row 4	3, 0	3, 1	3, 2	3, 3

Memory Address Calculation in an Array

Address Calculation in single (one) Dimension Array:

Array of an element of an array say "A[I]" is calculated using the following formula:

Address of A [I] = B + W * (I – LB)
Where,
B = Base address
W = Storage Size of one element stored in the array (in byte)
I = Subscript of element whose address is to be found
LB = Lower limit / Lower Bound of subscript, if not specified assume 0 (zero)

Address Calculation in Double (Two) Dimensional Array:
While storing the elements of a 2-D array in memory, these are allocated contiguous memory locations. Therefore, a 2-D array must be linearized so as to enable their storage. There are two alternatives to achieve linearization

a. Row-Major
b. Column-Major

Row Major System
The address of a location in Row Major System is calculated using the following formula:
Address of **A [I][J] = B + W * [N * (I – Lr) + (J – Lc)]**

Column Major System:
The address of a location in Column Major System is calculated using the following formula:
Address of A [I][J] Column Major Wise = B + W * [(I – Lr) + M * (J – Lc)]

Where,
B = Base address
I = Row subscript of element whose address is to be found
J = Column subscript of element whose address is to be found
W = Storage Size of one element stored in the array (in byte)
Lr = Lower limit of row/start row index of matrix, if not given assume 0 (zero)
Lc = Lower limit of column/start column index of matrix, if not given assume 0 (zero)
M = Number of row of the given matrix
Number of rows (M) will be calculated as = (Ur – Lr) + 1
Number of columns (N) will be calculated as = (Uc – Lc) + 1

(Problem Based on 1D and 2D Array: refer to class Notes).

Function

Function is a named group of programming statements, which perform a specific task and return a value.

1. **Built-in Functions (Library Functions)** :- The functions, which are already defined in C++ Library (in any header files) and a user can directly use these function without giving their definition is known as built-in or library functions. e.g., sqrt( ), toupper( ), isdigit( ) etc.
   Following are some important Header files and useful functions within them :

stdio.h (standard I/O function)	**gets( ) , puts( )**
ctype.h (character type function)	**isalnum( ) , isalpha( ), isdigit ( ), islower (), isupper ( ), tolower ( ), toupper( )**
string.h (string related function)	**strcpy ( ), strcat ( ), strlen( ), strcmp( ) , strcmpi( ) , strrev( ),strupr( ) , strlwr( )**
math.h (mathematical function)	**fabs ( ), pow ( ), sqrt ( ), sin ( ), cos ( ), abs ( )**
stdlib.h	**randomize ( ), random ( )**

randomize( ) : This function provides the seed value and an algorithm to help random( ) function in generating random numbers. The seed value may be taken from current system's time.

random(<int>) : This function accepts an integer parameter say x and then generates a random value between 0 to x-1. for example : random(7) will generate numbers between 0 to 6.

2. **User-defined function:** A function is a group of statements that together perform a task. Function always returns single value. By default function, return integer value.

C functions aspects	syntax
function definition	Return_type function_name (arguments list) { Body of function; }
function call	function_name (arguments list);
function declaration	return_type function_name (argument list);

- A **function declaration** tells the compiler about a function's name, return type, and parameters.
- A **function definition** provides the actual body of the function.
- When a function is called, control of the program gets transferred to the function.

Function Prototype

Function prototype tells compiler about number of parameters function takes, data-types of parameters and return type of function. By using this information, compiler cross check's function parameters and their data-type with function definition and function call. C++ has four types of prototypes.

1. No Return Type and No Arguments.
2. Return Type but No Arguments.
3. No Return Type but Arguments.
4. Return Type and Arguments.

Actual parameters : Actual parameters are parameters as they appear in function calls.
Formal parameters : Formal parameters are parameters as they appear in function definition.

There are three ways to passing a parameter to a function.

1. Call by value or pass by value
2. Call by reference or pass by reference
3. Call by pointer or pass by Pointer

Call by value or pass by value: In call by value, a copy of actual arguments is passed to formal arguments of the called function and any change made to the formal arguments in the called function have no effect on the values of actual arguments in the calling function.

Call by Reference or pass by Reference: In call by reference, a reference of actual arguments is passed to formal arguments of the called function and any change made to the formal arguments in the called function have effect on the values of actual arguments in the calling function.

Call by Reference or pass by Reference: In call by pointer, an address of actual arguments is passed to formal arguments of the called function and any change made to the formal arguments in the called function have effect on the values of actual arguments in the calling function.